

Intent-Router Cross-Check Gate

Test Suite & Verification Report — Dewlock, the intent-firewall copilot for agentic Sui DeFi

Feature · **Deterministic intent cross-check**

Commit · **91172d7**

Date · **2026-06-26**

Branch · **main**

STATUS · **57/57 PASS**

The moat, applied to intent. The LLM only *proposes* an action; a deterministic router re-derives the action category from the user's literal command and **blocks** any prepared trade whose `actionType` category does not match — catching LLM action-misdetection *before* a PTB ever reaches the user for signing. Same fail-closed posture as the Guardian.

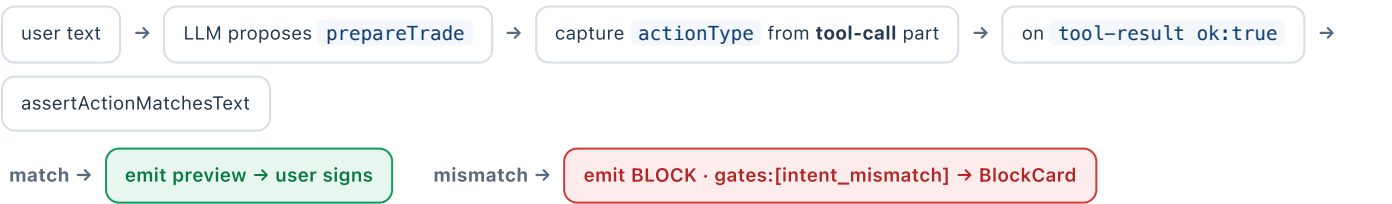
57 AUTOMATED TESTS	57 PASSING	0 FALSE POSITIVES	1103 FULL SUITE (ALL GREEN)	124ms SUITE RUNTIME
------------------------------	----------------------	-----------------------------	---------------------------------------	-------------------------------

1 · What is under test

Module `packages/agent/src/intent/intent-router.ts` (pure, deterministic, no LLM) exposes three units, wired into the agent stream at `apps/web/app/api/agent/route.ts`.

UNIT	RESPONSIBILITY	RETURNS
<code>routeAction(text)</code>	Classify the user's <i>literal</i> command into an action category from adapter keywords (swap / lend / stake / send) + guard-only verbs (bridge / limit). First value-verb of the first clause wins.	category · or <code>null</code> (read-only / no verb)
<code>categoryForActionType(at)</code>	Map a proposed <code>prepareTrade</code> <code>actionType</code> to its category. Composite & DeepBook-lifecycle return <code>null</code> (no single-verb mapping → not cross-checked).	category · or <code>null</code> (skip)
<code>assertActionMatchesText(text, at)</code>	The gate. Blocks only when both sides are non-null and differ. <code>null</code> on either side → pass (no false positive).	<code>{ ok, reason? }</code>

Server wiring — where the gate fires



Why a hard block matters: a misrouted action that reaches the preview is one click from signing the wrong on-chain transaction. The gate replaces the signable preview with a block card carrying the same `{ ok:false, reasons, gates }` shape the Guardian uses — so a misdetected trade can never be signed.

2 · Automated test cases (57)

2.1 · `routeAction` — value-verb classification (21 cases)

#	INPUT (USER TEXT)	EXPECTED	DONE CONDITION
1	swap 5 SUI to USDC	swap	swap keyword detected
2	sell 10 SUI for USDC	swap	<code>sell</code> is a swap synonym
3	lend 5 USDC on navi	lend	lend keyword detected

#	INPUT (USER TEXT)	EXPECTED	DONE CONDITION
4	deposit 100 USDC to suilend	<code>lend</code>	<code>deposit</code> → lend category
5	stake 10 SUI	<code>stake</code>	stake keyword detected
6	stake 5 SUI to afsui	<code>stake</code>	LST provider suffix ignored
7	send 1 SUI to alice	<code>send</code>	send keyword detected
8	transfer 2 USDC to 0xabc	<code>send</code>	<code>transfer</code> → send category
9	pay 1 SUI to bob	<code>send</code>	<code>pay</code> → send category
10	bridge USDC to ethereum	<code>bridge</code>	guard-only verb detected
11	redeem my bridge	<code>bridge</code>	<code>redeem</code> → bridge
12	limit order SUI at 3.50	<code>limit</code>	guard-only verb detected
13	show my portfolio	<code>null</code>	read-only verb → no action
14	what's the best yield	<code>null</code>	question → no action
15	check my balance	<code>null</code>	read-only verb → no action
16	portfolio	<code>null</code>	read-only noun → no action
17	where can I get the best APY?	<code>null</code>	question → no action
18	5 SUI USDC	<code>null</code>	no verb → ambiguous, no action
19	"" (empty)	<code>null</code>	empty → no action
20	" " (whitespace)	<code>null</code>	blank → no action
21	swap 5 SUI to USDC then lend it	<code>swap</code>	first clause wins (chaining handled upstream)

2.2 · `categoryForActionType` — `actionType` → `category` (17 cases)

#	ACTIONTYPE	EXPECTED	DONE CONDITION
22	swap	<code>swap</code>	1:1 map
23	lend_deposit	<code>lend</code>	all <code>lend_*</code> collapse to lend
24	lend_repay	<code>lend</code>	all <code>lend_*</code> collapse to lend
25	lend_borrow	<code>lend</code>	all <code>lend_*</code> collapse to lend
26	lend_withdraw	<code>lend</code>	all <code>lend_*</code> collapse to lend
27	stake	<code>stake</code>	1:1 map
28	unstake	<code>stake</code>	unstake shares stake category
29	transfer	<code>send</code>	transfer → send
30	limit_order	<code>limit</code>	1:1 map
31	bridge_redeem	<code>bridge</code>	1:1 map
32	composite	<code>null · skip</code>	multi-leg chain, no single verb
33	cancel_order	<code>null · skip</code>	DeepBook lifecycle, UI-driven
34	bm_create	<code>null · skip</code>	onboarding wizard, not NL
35	bm_deposit	<code>null · skip</code>	onboarding wizard, not NL
36	withdraw_settled	<code>null · skip</code>	order-management UI
37	claim_settled	<code>null · skip</code>	order-management UI
38	unknown_action	<code>null · skip</code>	unknown → skip (fail-open)

2.3 · `assertActionMatchesText` — the cross-check gate (19 cases)

#	USER TEXT	ACTIONTYPE	RESULT	DONE CONDITION
39	lend 5 USDC on navi	swap	BLOCK	reason names both "swap" & "lend" + "rephrase"
40	stake 10 SUI	lend_deposit	BLOCK	reason names "lend_deposit" & "stake"
41	send 1 SUI to alice	swap	BLOCK	send≠swap → blocked
42	swap 5 SUI to USDC	lend_deposit	BLOCK	swap≠lend → blocked
43	bridge USDC to eth	swap	BLOCK	bridge≠swap → blocked
44	swap 5 SUI to USDC	swap	PASS	category match → allowed
45	lend 5 USDC on navi	lend_deposit	PASS	both → lend
46	deposit 10 USDC to suilend	lend_repay	PASS	both → lend (sub-action differs, OK)
47	stake 10 SUI	stake	PASS	category match
48	stake 10 SUI	unstake	PASS	both → stake category (no over-block)
49	send 1 SUI to alice	transfer	PASS	both → send
50	show my portfolio & 2 more	swap · ...	PASS	read-only text → null → pass (no FP)
51	5 SUI USDC (no verb)	swap	PASS	ambiguous → null → pass (no FP)
52	"" (empty)	swap	PASS	empty → null → pass (no FP)
53	swap... / lend...	composite	SKIP	composite null → pass even if verb present
54	stake 10 SUI	cancel_order	SKIP	DeepBook lifecycle → pass
55	swap 5 SUI	bm_create	SKIP	onboarding → pass
56	stake 10 SUI	withdraw_settled	SKIP	order-mgmt → pass
57	lend 5 USDC on navi	swap	BLOCK	reason is user-safe: no "Error", contains "rephrase"

3 · Manual E2E test cases — transaction-type coverage

Run against the dev server (`pnpm --filter web dev → http://localhost:3000/app`), wallet connected. These exercise the full agent-stream path the unit tests stub out, across **every** transaction type Dewlock's `prepareTrade` can emit. Each row's done condition = the gate's correct verdict *plus* the expected card.

3.1 · Gate-CHECKED transactions — correct detection → PASS (value verbs; mismatch is hard-blocked, see §3.4)

#	TX TYPE	EXAMPLE COMMAND	ACTIONTYPE	EXPECTED CARD · DONE CONDITION
T1	Swap	swap 5 SUI to USDC	swap	Aggregator swap preview; route + min-out shown; sign enabled
T2	Swap (synonym)	sell 10 SUI for USDC · convert 5 SUI to USDC	swap	Same swap preview — <code>sell/convert/dump/exchange</code> all route to swap
T3	Lend deposit	lend 100 USDC on navi · supply 100 USDC	lend_deposit	NAVI deposit preview; supplied amount + new position
T4	Lend repay	repay 50 USDC on navi	lend_repay	Repay preview; debt reduction shown
T5	Lend borrow	borrow 20 USDC from navi	lend_borrow	Borrow preview + post-tx health-factor gate blocks if HF unsafe
T6	Lend withdraw	withdraw 30 USDC from navi	lend_withdraw	Withdraw preview; collateral change + HF check
T7	Stake afSUI	stake 10 SUI to afsui	stake	Aftermath afSUI mint preview; staking gate validates provider+target

#	TX TYPE	EXAMPLE COMMAND	ACTIONTYPE	EXPECTED CARD · DONE CONDITION
T8	Stake haSUI	stake 10 SUI to hasui	stake	Haedal direct-PTB haSUI preview; correct validator/object
T9	Unstake	unstake 5 afsUI	unstake	Unstake preview; both map to stake category (no over-block)
T10	Send → address	send 1 SUI to 0xabc...	transfer	Transfer preview; recipient = literal address
T11	Send → contact	pay 2 USDC to alice	transfer	Transfer preview; contact resolved from signed blob-book
T12	Limit order	limit buy SUI at 3.50	limit_order	DeepBook limit-order preview; price + size; limit category

3.2 · Gate-SKIPPED flows — `categoryForActionType → null` , never blocked (no single user-verb mapping)

#	FLOW	EXAMPLE	ACTIONTYPE	WHY SKIPPED · DONE CONDITION
T13	Atomic composite	swap 5 SUI to USDC then lend it → Run atomic	composite	Multi-leg recipe in 1 PTB; verified by the composite-recipe Guardian gate, not this one
T14	Sequential chain	multi-step plan (any combo), step-by-step	per-step	Each step re-submitted as a complete command → each step is gate-checked individually
T15	DeepBook onboarding	balance-manager wizard	bm_create bm_deposit	UI wizard, not NL-driven → skipped
T16	Cancel order	Cancel from an open-order card	cancel_order	Order-lifecycle UI action → skipped
T17	Withdraw / claim settled	Claim from a settled-funds card	withdraw_settled claim_settled	Order-management UI → skipped

3.3 · Read-only intents — `routeAction → null` , gate silent (no value verb)

#	EXAMPLE COMMAND	EXPECTED	DONE CONDITION
T18	show my portfolio	Portfolio card	Positions render; no gate involvement
T19	show my recent activity	History card	Activity list renders; gate silent
T20	what's the best yield for USDC	Advisor answer	Read-only yield advisor; nothing signable; gate silent

3.4 · Mismatch → BLOCK safety matrix — forced LLM misdetection (the load-bearing cases)

For each, the user's literal intent is unambiguous but the LLM proposes a *different* category. Expected: no signable preview — a **BlockCard** (`gates:[intent_mismatch]`) naming both actions + "rephrase".

#	USER TEXT (REAL INTENT)	LLM WRONGLY PROPOSES	VERDICT	DONE CONDITION
B1	lend 100 USDC on navi	swap	BLOCK	No swap preview; reason cites lend vs swap
B2	stake 10 SUI	lend_deposit	BLOCK	No lend preview; reason cites stake vs lend
B3	send 1 SUI to alice	swap	BLOCK	No swap preview — funds-leak intent stopped
B4	swap 5 SUI to USDC	stake	BLOCK	No stake preview; reason cites swap vs stake
B5	borrow 20 USDC from navi	swap	BLOCK	No swap preview; borrow≠swap
B6	repay 50 USDC on navi	transfer	BLOCK	No transfer preview; repay (lend)≠send
B7	withdraw 30 USDC from navi	swap	BLOCK	No swap preview; withdraw (lend)≠swap

3.5 · Detailed walkthroughs

M-1 · Block path

- 1 Type `lend 100 USDC on navi` ; LLM (misdetecting) proposes `prepareTrade(actionType:"swap")` .
- 2 **Expect:** no signable swap preview — a **BlockCard** with *"Refusing for safety: the prepared action (|\"swap|") doesn't match what you asked (|\"lend|")..."*, gate `intent_mismatch` .
- 3 **Done when:** nothing signable shown; rephrase prompt; no wallet popup.

M-2 · Pass path

- 1 Type `swap 5 SUI to USDC` ; LLM proposes `actionType:"swap"` .
- 2 **Done when:** normal Guardian-checked swap preview renders; gate adds zero friction to a correct turn.

M-3 · No-false-positive

- 1 `show my portfolio` → portfolio renders, gate silent (`routeAction` null).
- 2 Atomic `swap 5 SUI to USDC then lend it` → composite skipped (`categoryForActionType` null); chain proceeds.
- 3 **Done when:** neither legitimate flow is blocked.

4 · Verification checklist · done conditions

- ✓ **Unit coverage:** 57/57 automated cases pass (`vitest run intent-router.test.ts` — 124ms).
- ✓ **Full regression:** 1103/1103 tests pass — zero regressions from the gate (1046 prior + 57 new).
- ✓ **Typecheck:** agent + web typecheck clean; the new module emits no errors.
- ✓ **Runtime resolution:** `require("@dewlock/agent/intent/intent-router")` resolves from `apps/web` (where the route runs); package export added & dist built.
- ✓ **Behavior smoke:** `routeAction("lend 5 USDC on navi") = "lend"` ; mismatch vs `swap` returns the safe block reason at runtime.
- ✓ **Block shape:** emitted `{ ok:false, reasons, gates:["intent_mismatch"] }` matches the Guardian block shape → renders via `isPrepareTradeResult` → `makeBlockCard` → `BlockCard` (verified end-to-end).
- ✓ **No false positives:** null on either side passes — read-only, ambiguous, empty, composite, DeepBook all flow through.
- ✓ **Fail-open machinery:** router throw / module-load failure / uncapturable `actionType` → gate skips; only a positive category mismatch is a hard block.
- ✓ **Dev server:** rebuilt agent dist picked up; `/app` returns HTTP 200.
- ✓ **Committed:** 91172d7 on `main` (not pushed).

Known limitation (fail-open, by design): the gate reads `actionType` from the Mastra **tool-call** stream part because the `ok:true` result carries no `actionType` . It relies on the tool-call part arriving before the tool-result in Mastra 1.42's `fullStream` (it does). If that ordering ever changes, the captured `actionType` is null and the gate **fails open** — safe (never blocks a legit turn) but blind. Hardening follow-up: add `actionType` to the `prepareTrade` output schema for a result-side fallback.